LABORATORIO DE PROGRAMACIÓN SEGURA I · INACAP LA SERENA

Manual de implementación Carta Digital PNK

Despliegue completo en AWS con alta disponibilidad (ALB + Auto Scaling + RDS + EFS) y Cloudflare, con un solo script.

Versión del manual	Tiempo de despliegue	Repositorio
1.0 · 12-11-2026	**≈ 9-12 minutos**	**matiasNon/pnkSecurity**

Contenido

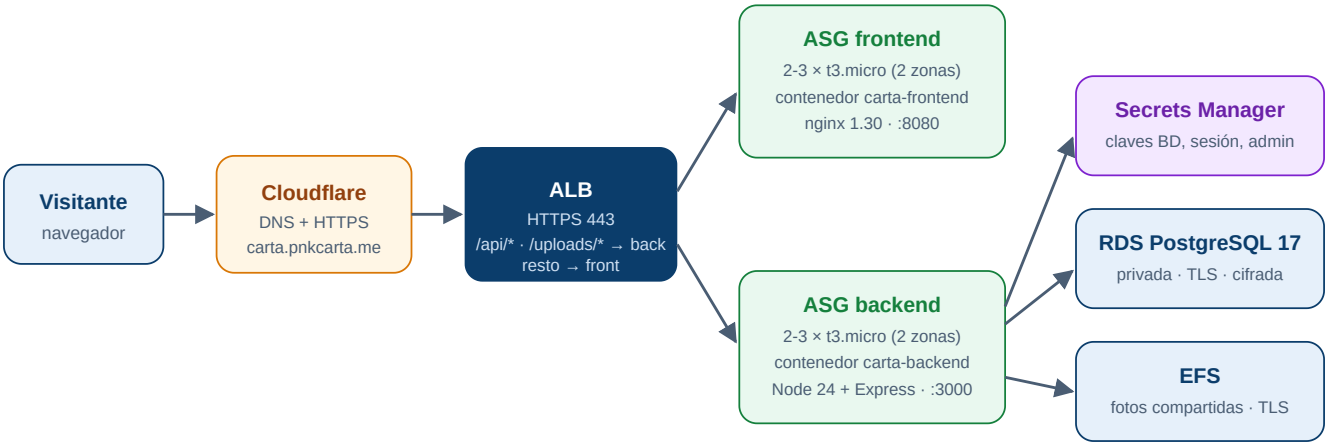
1. Resumen y requisitos previos
2. Arquitectura
3. Componentes, contenedores y versiones
4. Despliegue en 5 pasos (script único)
5. Cloudflare: DNS y HTTPS (paso manual)
6. Verificación del ambiente
7. Actualizar la aplicación sin cortar el servicio
8. Procedimiento de parcheo ante vulnerabilidades
9. Borrar el ambiente y volver a desplegar
10. Solución de problemas
11. Seguridad del despliegue

1. Resumen y requisitos previos

Todo el ambiente (red, base de datos, almacenamiento, secretos, balanceador, instancias y contenedores) se crea con `infra/aws/desplegar.sh` sobre la plantilla CloudFormation `infra/aws/carta-digital.yaml`. Solo la configuración de Cloudflare es manual, como pide el requisito.

REQUISITO	DETALLE
Cuenta AWS	AWS Academy Learner Lab (región **us-east-1**), con el laboratorio iniciado (*Start Lab*, luz verde). Usa el rol `LabRole` y el perfil `LabInstanceProfile`.
Terminal	**AWS CloudShell** (ícono `>_` en la consola de AWS). Ya trae AWS CLI, git, openssl y jq; no hay que instalar nada.
Rama del código	`front-back-separados` (frontend y backend separados con la infraestructura AWS). Cuando `produccion` se actualice a esa rama, se puede usar `produccion`.
Dominio (opcional)	`pnkcarta.me` administrado en Cloudflare (sección 5). Sin dominio la app funciona en la dirección del ALB.

2. Arquitectura



Las instancias no tienen IP pública ni SSH; solo el ALB llega a ellas. La base de datos y EFS solo aceptan conexiones desde el Security Group del backend. Las sesiones y los contadores anti fuerza bruta viven en PostgreSQL, así que cualquier instancia atiende cualquier petición.

3. Componentes, contenedores y versiones

3.1 Contenedores

CONTENEDOR	IMAGEN Y ORIGEN	UBICACIÓN EN LA INSTANCIA	PUERTO
carta-backend	`carta-backend:latest`, construida en cada instancia desde `Dockerfile` (etapa `runtime`)	Código: `/opt/carta-digital` · fotos: `/mnt/efs/uploads` (EFS) · CA de RDS: `/opt/carta-digital/rds-ca.pem` · log de arranque: `/var/log/carta-backend-arranque.log`	3000
carta-frontend	`carta-frontend:latest`, desde `frontend/Dockerfile` (etapa `aws`)	Código: `/opt/carta-digital` · log: `/var/log/carta-frontend-arranque.log`	8080

Ambos se definen en `docker-compose.aws.yml` (perfiles `backend` y `frontend`; proyecto Docker `carta-digital`) y corren con usuario no root, sistema de archivos de solo lectura, `cap_drop: ALL` y `no-new-privileges`.

3.2 Versiones

COMPONENTE	VERSIÓN	DÓNDE SE FIJA
Node.js (backend)	24 (imagen `node:24-alpine`; hoy 24.22)	`Dockerfile`
Express / express-session	4.22 / 1.19	`package.json` + `package-lock.json`
pg / connect-pg-simple	8.23 / 10.0	idem
helmet / zod / multer / pino	8.3 / 4.6 / 2.5 / 9.14	idem
nginx (frontend)	1.30.6 (`nginxinc/nginx-unprivileged:1.30-alpine`)	`frontend/Dockerfile`

PostgreSQL (Amazon RDS)	17 (menor vigente, p. ej. 17.9; `db.t3.micro`)	`desplegar.sh` consulta la menor por defecto
Sistema operativo de las instancias	Ubuntu Server 24.04 LTS (última AMI de Canonical)	Parámetro `AmiId` (SSM)
Docker / AWS CLI	Docker CE y Compose (repositorio oficial) / AWS CLI v2	user-data de la plantilla

3.3 Parámetros de la plantilla (valores por defecto)

PARÁMETRO	DEFECTO	USO
`Rama`	`produccion`	Rama que clona cada instancia (se pasa con `RAMA=`)
`DominioPublico`	vacío	Dominio para redirecciones (se pasa con `DOMINIO=`)
`TipoInstancia` / `ClaseBd`	`t3.micro` / `db.t3.micro`	Tamaño de instancias y BD
Frontend y backend: mín / máx	2 / 3 cada uno	Escalado por CPU (objetivo 60 %)

4. Despliegue en 5 pasos (script único)

1 Abrir CloudShell

En el Learner Lab: *Start Lab* → esperar la luz verde → botón **AWS**. En la consola, región **N. Virginia (us-east-1)** → ícono `>_` (CloudShell).

2 Clonar el repositorio

```
git clone --branch front-back-separados https://github.com/matiasNon/pnkSecurity.git
cd pnkSecurity
```

3 Ejecutar el script

```
RAMA=front-back-separados DOMINIO=carta.pnkcarta.me bash infra/aws/desplegar.sh
```

El script muestra 5 etapas: credenciales → red (VPC por defecto y 2 subredes en zonas distintas) → certificado HTTPS importado a ACM → stack CloudFormation → espera a que el balanceador vea las 4 instancias sanas. Tarda **unos 9-12 minutos** (RDS es lo más lento).

> **Importante:** la variable `RAMA` es obligatoria mientras `produccion` no tenga esta versión. Sin ella, las instancias clonarían código antiguo sin la infraestructura separada.

4 Anotar la dirección del ALB

Al final el script imprime la URL y deja todo en `infra/aws/salida.env` (sin secretos):

```
cat infra/aws/salida.env      # ALB_DNS=carta-digital-alb-XXXX.us-east-1.elb.amazonaws.com
```

5 Apuntar Cloudflare al ALB

Editar el CNAME `carta` en Cloudflare con el nuevo `ALB_DNS` (sección 5). Es el único paso manual.

Clave del administrador

Se genera al azar y se guarda en Secrets Manager (nunca en el código). Para verla:

```
ARN=$(aws cloudformation describe-stacks --stack-name carta-digital \
  --query "Stacks[0].Outputs[?OutputKey=='SecretoAdminArn'].OutputValue" --output text)
aws secretsmanager get-secret-value --secret-id "$ARN" --query SecretString --output text | jq -r .password
```

Usuario: `admin@gmail.com`. Panel: `https://carta.pnkcarta.me/admin`.

5. Cloudflare: DNS y HTTPS (paso manual)

#	DÓNDE	QUÉ HACER
1	Cloudflare → DNS → Records	CNAME `carta` → `ALB_DNS`, **Proxied** (nube naranja). Si ya existe, solo editar el destino.
2	Namecheap → Manage → Nameservers	Solo la primera vez: **Custom DNS** con `carter.ns.cloudflare.com` y `lila.ns.cloudflare.com`.
3	SSL/TLS → Overview	Modo **Full** (no "Full (strict)": el ALB usa un certificado autofirmado y daría error 526).
4	SSL/TLS → Edge Certificates	**Always Use HTTPS: ON** y **Minimum TLS Version: TLS 1.2**.

Guía detallada y verificación: `docs/CLOUDFLARE.md`.

6. Verificación del ambiente

```
curl -sI https://carta.pnkcarta.me/               # 200, server: cloudflare, strict-transport-security
curl -s  https://carta.pnkcarta.me/api/health     # {"estado":"ok","servicio":"backend"}
curl -s  https://carta.pnkcarta.me/api/estado     # baseDatos.tls = true, minimoPrivilegio = true
curl -s -o /dev/null -w "%{http_code}\n" https://carta.pnkcarta.me/api/admin/productos   # 401 sin sesión
```

DIRECCIÓN	QUÉ SE VE
`/restaurantes/1`	Carta pública
`/restaurantes/1/carrito`	Carrito (agregar, cambiar cantidad, quitar)
`/admin`	Panel de administración (CRUD de cartas, categorías y productos)
`/api/`	Consola del backend: estado en vivo de BD, EFS, balanceo entre instancias y CRUD
`/privacidad`	Política de privacidad y "Mis datos" (acceso, rectificación, supresión)

> En la consola `/api/` deben aparecer **dos instancias distintas** turnándose en "Balanceo de carga en vivo", "Cifrada con TLS ✓" en la base de datos y "Mínimo privilegio ✓" en el usuario de la app.

7. Actualizar la aplicación sin cortar el servicio

Cada instancia clona la rama al arrancar. Para publicar un cambio ya subido a GitHub, se reemplazan las instancias de a una:

```
cd ~/pnkSecurity && git pull
bash infra/aws/actualizar.sh             # backend y luego frontend (≈ 10 min)
bash infra/aws/actualizar.sh backend     # solo una capa (≈ 5 min)
```

Se usa en vez de *instance refresh* porque AWS Academy lo bloquea. Si cambió la **plantilla** (`carta-digital.yaml`), primero se vuelve a ejecutar `desplegar.sh` con las mismas variables (actualiza el stack sin borrar datos) y después `actualizar.sh` .

La pipeline de Jenkins (`Jenkinsfile`) hace lo mismo automáticamente desde la rama `produccion` , solo si pasan lint, SAST, las pruebas Jest y Playwright y el filtro de seguridad (Gitleaks, npm audit + OSV, librerías deprecadas, OWASP Dependency-Check, Trivy y OWASP ZAP).

8. Procedimiento de parcheo ante vulnerabilidades

QUÉ SE PARCHA	CÓMO SE DETECTA	CÓMO SE CORRIGE Y DESPLIEGA
Dependencias de Node (npm)	`npm audit --omit=dev` , OSV-Scanner, OWASP Dependency-Check (pipeline)	`npm update <paquete>` o subir la versión en `package.json` → `npm install` → pruebas (`scripts/qa.sh`) → commit y push → `actualizar.sh backend`
Paquetes del sistema de las imágenes (Alpine)	Trivy (pipeline) sobre `carta-backend` y `carta-frontend`	Las imágenes se reconstruyen en cada instancia nueva y el frontend ejecuta `apk upgrade` al construirse: basta con `actualizar.sh`
Versión de nginx o de Node	Trivy marca el paquete `nginx` o `node`	Cambiar la etiqueta base (p. ej. `1.30-alpine` → siguiente estable) en el Dockerfile → pruebas → push → `actualizar.sh`
Sistema operativo de las instancias	Boletines de Ubuntu	Las instancias nuevas usan la última AMI de Ubuntu 24.04 (parámetro SSM): `actualizar.sh` las reemplaza ya parchadas
PostgreSQL (RDS)	Avisos de RDS / versión menor nueva	Consola RDS → Modificar → versión menor 17.x (aplicar en la ventana de mantenimiento)
Secretos expuestos	Gitleaks (pipeline)	Rotar en Secrets Manager y reemplazar las instancias (`actualizar.sh`); el rol de la BD se re-aplica en cada arranque

> **Reversión:** `git revert <commit>` → push → `actualizar.sh` . Como el reemplazo es de a una instancia, el sitio sigue atendiendo durante el parche y la reversión.

9. Borrar el ambiente y volver a desplegar

```
bash infra/aws/destruir.sh                # pide escribir BORRAR; ≈ 6-8 min
RAMA=front-back-separados DOMINIO=carta.pnkcarta.me bash infra/aws/desplegar.sh   # ≈ 9-12 min
```

`destruir.sh` elimina el stack completo (instancias, ALB, RDS sin respaldo final, EFS, secretos, Security Groups), el grupo de registros de CloudWatch y el certificado de ACM. Después de volver a desplegar, **el ALB tiene otro nombre**: actualizar el CNAME `carta` en Cloudflare (sección 5, paso 1). La base vuelve a los datos iniciales de las migraciones.

10. Solución de problemas

SÍNTOMA	CAUSA PROBABLE	QUÉ HACER
503 justo después de desplegar o de *Start Lab*	Las instancias están instalando Docker, construyendo imágenes o esperando a RDS	Esperar 10-20 min; en EC2 → Target Groups se ve cuándo quedan *healthy*
CloudShell: `ExpiredToken` o acceso denegado	Sesión del laboratorio vencida (≈ 4 h)	Volver a abrir la consola desde el botón AWS del Learner Lab
El dominio da error 522/523	CNAME apuntando a un ALB que ya no existe	Actualizar el CNAME con `ALB_DNS` de `salida.env`
Error 526 en Cloudflare	SSL en modo "Full (strict)"	Cambiar a "Full"
No se puede iniciar sesión	Se entró por `http://` o por otra dirección	Usar `https://carta.pnkcarta.me` (la cookie es *Secure*)
Consola: "NFS sin TLS" o "Con privilegios de administración"	Se activó un respaldo automático en el arranque	EC2 → instancia del backend → Actions → Monitor and troubleshoot → **Get system log** y buscar las líneas "AVISO" (es la misma salida de `/var/log/carta-backend-arranque.log`)
`desplegar.sh` falla en el stack	Permiso no disponible en el laboratorio o nombre en uso	CloudFormation → Events muestra el recurso; corregir y ejecutar de nuevo (es idempotente)

11. Seguridad del despliegue

CONTROL	IMPLEMENTACIÓN
Cifrado en tránsito	Cloudflare (Let's Encrypt) → ALB HTTPS → RDS con TLS verificado contra la CA de AWS; EFS por túnel TLS; HSTS
Cifrado en reposo	RDS, EFS y discos EBS cifrados
Red	Security Groups por capa: solo el ALB llega a las instancias; solo el backend llega a RDS y EFS; sin SSH

Secretos	Secrets Manager (5 secretos generados al azar); el archivo de entorno vive solo en la instancia (permisos de root)
Mínimo privilegio	La app usa el rol `carta_app` (solo lectura y escritura de datos); el usuario maestro solo migra al arrancar
Instancias	IMDSv2 obligatorio con salto 1 (los contenedores no leen las credenciales de la instancia)
Registros	CloudWatch Logs `/carta-digital/backend` con retención de 30 días
Evidencia	`docs/MATRIZ_ASVS_ISO.md`, `docs/REVISION_SSDLC_PRIVACIDAD.md`, `docs/PENTEST_OWASP_ASVS5.md`

Carta Digital PNK · Manual de implementación v1.0 · Generado desde `docs/manual/MANUAL_IMPLEMENTACION.html` del repositorio.